

Learned Burst-Time Prediction for Shortest-Remaining-Time-First CPU Scheduling: An Empirical Study on a Production Cluster Trace

Dheerajkumar Sasikumar
B.Tech. Artificial Intelligence and Machine Learning
Vellore Institute of Technology
dheerajsm2007@gmail.com

Abstract—Shortest-remaining-time-first (SRTF) scheduling minimises mean waiting time on a single processor only when the service demand of every task is known in advance; operating systems approximate that demand by exponential averaging of past bursts. This paper asks how much of the gap between exponential-averaging SRTF and clairvoyant (oracle) SRTF can be recovered by replacing the estimator with a supervised model trained on features observable when a task arrives. Using the first 10,000 tasks of the Alibaba cluster-trace-v2018 batch workload, a leakage-safe feature set was constructed with a causal event timeline, six predictors (exponential averaging, last-two averaging, a median baseline, linear regression, gradient-boosted trees and an oracle) were evaluated on a chronological split, and each was embedded in a discrete-event single-CPU simulator alongside first-come-first-served and round-robin baselines. The gradient-boosted predictor recovered 58.5% of the waiting-time gap between exponential-averaging SRTF and oracle SRTF, yet no prediction-driven scheduler outperformed round robin on this workload. A sensitivity analysis with bounded-uniform and log-normal prediction noise locates the error level at which SRTF loses its advantage over round robin and shows that the errors of the fitted models are structured rather than random: the gradient-boosted model waits about 2.9 times longer than random error of the same spread would imply, because it systematically under-predicts the rare very long tasks. Mean absolute error does not predict scheduling quality; rank agreement with the true runtime does. All tables, figures and numbers are generated programmatically from the experiment outputs.

Index Terms—CPU scheduling, shortest remaining time first, runtime prediction, machine learning, cluster traces, leakage-safe evaluation

I. INTRODUCTION

A. Motivation

Shortest-remaining-time-first (SRTF), the preemptive form of shortest-job-first, minimises mean waiting time on a single processor [1]. It is also taught as a policy that cannot be implemented exactly, because the length of the next CPU burst is unknown when the scheduling decision is made [2]. The textbook remedy is to estimate that length by exponential averaging of previous bursts. Meanwhile, learned components have displaced hand-designed heuristics elsewhere in computer systems, including index structures [3], cache replacement [4]–[7] and compiler optimisation [8]. It is therefore natural to ask whether a supervised model, trained only on information available when a task arrives, is a better estimator

for SRTF, and how much of SRTF’s theoretical advantage such an estimator actually delivers.

B. The Size and Location of the Gap

On the workload studied here, the room for a better estimator is large. Oracle SRTF, which is given every task’s true runtime, achieves a mean waiting time of 5.78 h per task, while SRTF driven by exponential averaging waits 63.2 h, a factor of 10.9 worse (Table III). The same table shows that round robin, which uses no runtime estimate at all, waits 11.5 h, which is 5.48 times less than exponential-averaging SRTF. The question that matters to a scheduler designer is therefore not whether machine learning can beat exponential averaging. It is whether any predictor restricted to arrival-time information can make SRTF competitive with a policy that needs no prediction at all.

C. Approach and Principal Findings

This paper answers that question empirically on the first 10,000 tasks of the Alibaba cluster-trace-v2018 batch workload [9]. Six predictors, from exponential averaging to gradient-boosted trees, are trained on leakage-safe features and evaluated on a chronological split, and each one drives SRTF in a discrete-event single-CPU simulator. The gradient-boosted predictor recovers 58.5% of the waiting-time gap between exponential-averaging SRTF and oracle SRTF, yet it still waits 2.56 times longer than round robin, and no prediction-driven SRTF outperforms round robin.

This negative result is the principal finding, not a caveat. On this trace the features available at arrival do not carry the information SRTF needs most, namely which of the arriving tasks are the rare very long ones. A two-model sensitivity analysis shows why the fitted models fall short of what their aggregate error suggests: their errors are concentrated on exactly those long tasks.

D. Contributions

- 1) A leakage-safe feature pipeline and set of online baselines on a real production trace, with unit tests that fail if any task still running at another task’s arrival contributes to that task’s features or predictions.

- 2) An end-to-end mapping from prediction error to scheduling outcome. The recovered fraction of the gap, (5), and rank agreement are reported next to conventional error metrics, and the results show that mean absolute error does not predict scheduling quality.
- 3) A two-model sensitivity analysis, bounded uniform and log-normal, that locates the error level at which SRTF loses to round robin and shows that structured error costs far more waiting time than random error of the same spread.
- 4) A reproducible artefact in which every table, figure and number in the text is generated from the experiment outputs.

E. Organisation

Section II reviews related work. Section III formalises the scheduling and prediction problem, Section IV describes the implementation and Section V the experimental setup. Section VI presents and interprets the results, Section VII discusses threats to validity and Section VIII concludes.

II. RELATED WORK

A. Burst-Time Prediction for CPU Scheduling

Predicting resource demand to inform scheduling predates machine learning. Dinda [10] built a real-time scheduling advisor on predicted running times. The Network Weather Service line of work forecast network performance and CPU availability from time series [11]–[13], and Zhang et al. [14] predicted task running times in grids from CPU-load predictions. Helmy et al. [15] framed CPU-burst estimation as supervised learning, training nearest-neighbour, support-vector, neural-network and decision-tree models on 16 process features from the GWA-T-4 grid trace. Jha and Goyal [16] pursued the same goal, and Pourali et al. [17] used a fuzzy rather than a learned model. Effah et al. [18] is the closest prior work to this paper: it predicts burst times to drive SJF and SRTF, but on a synthetic dataset modelled on GWA-T-4. Shafin and Ahmed [19] revisited the Helmy formulation with a strict pre-submission feature policy and chronological baselines, and this paper adopts their leakage-safe framing. The present work differs from this line in two respects. It uses a real production trace under a chronological split, and it reports the waiting time that prediction accuracy buys, not prediction accuracy alone.

B. Runtime Prediction for HPC Backfilling

The richest neighbouring literature concerns runtime estimates for backfilling batch schedulers. User-requested runtimes are known to be poor estimates [20], [21], and Chiang et al. [22] showed that more accurate requested runtimes improve production scheduling performance, which is the premise behind the recovery metric of this paper. Tsafirir et al. [23] replaced user estimates with system-generated predictions from each user’s last two jobs, the origin of the Last2 baseline used here, and Gaussier et al. [24] used online regression for the same purpose. Later work applied ensembles, transformers and

automata-based deep models [25]–[27], predicted from system logs [28], and predicted other job resources [29]. Because an under-estimate can get a job killed at its walltime limit, a separate strand targets under-estimation directly, through Tobit adjustment [30], dedicated predictors [31], soft walltimes [32] and uncertainty-aware quantile prediction [33]. TARE [34] argues that mean-based metrics hide tail behaviour, which is why this paper reports the under-prediction rate separately.

Single-CPU SRTF is a harsher consumer of predictions than backfilling. Tsafirir et al. correct a prediction once a job outlives it [23], whereas the prediction-driven SRTF of (2) fixes the estimate at arrival and never revises it. An under-predicted long task is therefore dispatched ahead of many short ones and, once its predicted remaining time is exhausted, cannot be preempted.

C. Learned Scheduling Policies

A different line of work learns the scheduling policy itself with reinforcement learning. DeepRM [35] introduced the approach for multi-resource cluster scheduling, and later variants refined its state and action design [36]–[39]. Decima [40] learned schedules for data-processing clusters and reported large reductions in job completion time over hand-tuned heuristics, and RLScheduler [41] applied the approach to HPC batch scheduling. This paper deliberately does not learn the policy. It keeps SRTF, which is optimal given true runtimes, and learns only its input. The oracle then gives a known ceiling, and the part of the gap that is due to the predictor can be measured directly, which an end-to-end learned policy does not provide.

D. Cluster Traces

Public production traces underpin this line of research. The Google traces [42]–[45] document the Borg cluster manager’s workload [46]. The Alibaba traces capture co-located online services and batch jobs [9], [47]–[50]. This study uses the batch-task table of the 2018 Alibaba release.

III. PROBLEM FORMULATION

A. Scheduling Model

A single processor serves a finite set of n tasks. Task i becomes runnable at its arrival time a_i and requires t_i units of processor time, its *runtime*. The scheduler may preempt the running task at any decision point; preemption cost is not modelled. Let S_i denote the instant at which task i first receives the processor and C_i its completion instant. The per-task metrics are the waiting time $W_i = C_i - a_i - t_i$, the turnaround time $T_i = C_i - a_i$ and the response time $R_i = S_i - a_i$. The objective is the mean waiting time $\bar{W} = \frac{1}{n} \sum_i W_i$.

B. Shortest Remaining Time First

Let $e_i(\tau)$ be the processor time that task i has received by instant τ and $\mathcal{R}(\tau)$ the set of runnable, incomplete tasks. At every arrival and every completion, SRTF selects

$$i^*(\tau) = \arg \min_{i \in \mathcal{R}(\tau)} (t_i - e_i(\tau)), \quad (1)$$

the task with the least remaining service. When every t_i is known at a_i , this rule minimises $\bar{W}$ on a single processor [1]. No operating system knows t_i in advance, so (1) is a ceiling rather than an implementable policy; this paper refers to it as *oracle SRTF*.

C. Prediction-Driven SRTF

A prediction-driven scheduler replaces t_i in (1) by an estimate $\hat{t}_i = f(\mathbf{x}_i)$ computed once, when task i arrives, from a feature vector $\mathbf{x}_i$, and never revised afterwards:

$$i^*(\tau) = \arg \min_{i \in \mathcal{R}(\tau)} (\hat{t}_i - e_i(\tau)). \quad (2)$$

The textbook estimator is exponential averaging [2], which maintains a running estimate τ_n from the sequence of observed runtimes $t_1, t_2, \dots$ of the same source:

$$\tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n, \quad 0 \leq \alpha \leq 1. \quad (3)$$

In this work the source is the job identifier: one estimate is maintained per job, updated at each completion of one of its tasks, and seeded with the first observed runtime of that job. A task whose job has no completed history is assigned the running mean of all completed runtimes.

D. Prediction Target and Admissible Features

The prediction target is the *job-level runtime* recorded by the trace: the wall-clock interval between the recorded start and end of a scheduled unit of work (in the Alibaba trace, a *task*, which is one stage of a job executed by many instances). It is not the CPU burst between two I/O operations that (3) was designed to estimate, and the paper makes no claim to predict such bursts; the simulator treats this runtime as the service demand t_i in (1).

A feature is admissible only if its value is fixed at a_i . Historical features of task i are computed from

$$\mathcal{H}_i = \{j : C_j < a_i\}, \quad (4)$$

the set of tasks that completed *strictly* before task i arrived, following the leakage-safe framing of Shafin and Ahmed [19]. A task that arrived earlier but is still running at a_i is excluded, and no train/test split is ever random.

E. Evaluation Quantities

Prediction quality is reported as mean absolute error, root mean squared error, and the under-prediction rate, the fraction of tasks with $\hat{t}_i < t_i$. The last is reported separately because aggregate error metrics hide tail behaviour [34], and because under- and over-estimation of equal magnitude have very different consequences under (2): an under-estimated task's predicted remaining service becomes negative as it runs and it can no longer be preempted. Because (2) consumes only the *order* of the predictions, the Spearman rank correlation ρ_s between predicted and actual runtime is reported as well.

Let $\bar{W}_{\text{EA}}$, $\bar{W}_{\text{GBM}}$ and $\bar{W}_{\text{OR}}$ denote the mean waiting time of SRTF driven by exponential averaging, by the gradient-boosted predictor and by the oracle. The headline quantity is the fraction of the gap that the learned predictor recovers,

$$\text{recovery} = \frac{\bar{W}_{\text{EA}} - \bar{W}_{\text{GBM}}}{\bar{W}_{\text{EA}} - \bar{W}_{\text{OR}}} \times 100\%. \quad (5)$$

IV. METHODOLOGY

The study was implemented as a five-stage pipeline: trace preprocessing, causal feature construction, predictor training, discrete-event simulation, and experiment orchestration. Each stage is described as implemented.

A. Trace Preprocessing

The `batch_task` table was read in chunks of 500 000 rows so that the raw file was never resident in memory. Rows with a missing start or end timestamp, a non-positive runtime, or a status other than `Terminated` were discarded, in that order. The runtime was defined as $t_i = \text{end_time}_i - \text{start_time}_i$. The table records no submission timestamp, so the arrival time was set to $a_i = \text{start_time}_i$; the consequences of this proxy are discussed in Section VII. The first N tasks in ascending order of (arrival time, job name, task name) were retained by maintaining a bounded buffer of the earliest candidates across chunks. No random sampling was performed, because the chronological split of Section V depends on this ordering.

B. Causal Feature Construction

Historical features were computed in a single pass over a merged timeline of $2N$ events, one arrival and one completion per task, sorted by time with arrivals ordered before completions at equal timestamps so that (4) holds with strict inequality. An arrival event reads the current state and fixes the task's features; a completion event appends the task's runtime to its job's history and to a global running sum. Three historical features result: `previous_runtime`, the most recently completed same-job runtime; `rolling_mean_3`, the mean of up to the three most recently completed same-job runtimes; and `has_history`, a flag that is false when the job has no completed task yet. When the flag is false both numeric features take the running mean of all completed runtimes; when no task at all has completed they are undefined, and such rows were excluded from the fitting of the regression models. Direct features were taken from the task's own record or derived from a_i alone. A unit test asserts that a same-job task which arrived earlier but completes after a_i does not contribute, and that the same test fails under a conventional grouped shift over the arrival-sorted table.

C. Predictors

Six predictors share the interface `fit( $X_{\text{train}}, y_{\text{train}}$ ), predict( $X$ )`.

- *Exponential averaging* implements (3) per job.
- *Last2* predicts the mean of the two most recently completed same-job runtimes and falls back to the running global mean, following Tsafirir et al. [23].

- *Median* predicts the training-set median runtime for every task; it is the constant reference that any learned model must beat.
- *Linear regression* is ordinary least squares on $\log(1 + t_i)$; the right-skewed inputs `n_instances`, `previous_runtime` and `rolling_mean_3` are transformed by $\log(1+x)$ and all inputs are standardised.
- *Gradient boosting* is XGBoost [51] on $\log(1 + t_i)$ with the configuration of Section V.
- *Oracle* returns t_i itself and defines the ceiling.

The logarithmic target was adopted because the runtime spans five orders of magnitude and, in preliminary runs, squared-error regression on the raw scale produced negative predictions for ordinary tasks. Predictions of both regression models are mapped back with $\exp(\cdot) - 1$ and floored at 1 s, the smallest runtime in the data. Exponential averaging and Last2 are sequential estimators. `fit` stores the training events, and `predict` walks one merged timeline of training and test arrivals and completions from an empty state, as an online scheduler would. Each completion updates the state only when it occurs, and each test prediction is fixed at the task’s arrival. A training task still running when a test task arrives has therefore not yet been observed, and a unit test asserts this across the train/test boundary.

D. Discrete-Event Simulator

A single-CPU simulator was written with one event loop per policy: first-come-first-served (non-preemptive), round robin with a fixed quantum, and predictor-driven SRTF. Algorithm 1 gives the SRTF loop as implemented. Predictions are obtained once for the whole task set by a single call to the predictor and thereafter treated as constants. The loop tracks, for the running task r , both the true remaining service ρ_r , which determines when the task actually finishes, and the predicted remaining service $\hat{t}_r - e_r$, which alone determines scheduling decisions. Re-evaluation occurs at every arrival and every completion, and a slice runs until the earlier of the running task’s true completion and the next arrival. A context switch is counted whenever the task starting a slice differs from the task that ran in the preceding slice; the first slice is not counted. When the predictor is the oracle, $\hat{t}_i = t_i$ and Algorithm 1 reduces to (1); a unit test asserts per-task equality of first-dispatch, completion, waiting, turnaround and response times between this configuration and an independently written classical SRTF on a hand-worked four-process example and on five randomised instances.

E. Experiment Orchestration

A driver script fits every predictor on the training split, runs the seven schedulers over the test split, writes the comparison table, aborts if the oracle-driven SRTF is not the minimum-waiting-time scheduler, runs the two sensitivity sweeps, and computes (5). A second script converts the result tables into the LaTeX tables and numeric macros of this paper, so that no reported number is transcribed by hand.

Algorithm 1 Predictor-driven SRTF on a single CPU (as implemented)

Require: tasks (a_i, t_i) sorted by (a_i, id) ; predictions $\hat{t}_i = f(\mathbf{x}_i)$ fixed at arrival

- 1: $\mathcal{R} \leftarrow \emptyset$; $r \leftarrow \perp$; $\tau \leftarrow a_1$; $e_i \leftarrow 0$ and $\rho_i \leftarrow t_i$ for all i
- 2: **while** some task is incomplete **do**
- 3: admit every unadmitted i with $a_i \leq \tau$ into $\mathcal{R}$
- 4: **if** $\mathcal{R} = \emptyset$ and $r = \perp$ **then**
- 5: $\tau \leftarrow$ next arrival time; **continue**
- 6: **end if**
- 7: $c \leftarrow \arg \min_{i \in \mathcal{R} \cup \{r\}} (\hat{t}_i - e_i, a_i, i)$ {lexicographic}
- 8: **if** $c \neq r$ **then**
- 9: **if** $r \neq \perp$: return r to $\mathcal{R}$
- 10: remove c from $\mathcal{R}$; record $S_c \leftarrow \tau$ on first dispatch;
- 11: $r \leftarrow c$
- 12: $\delta \leftarrow \min(\rho_r, a_{\text{next}} - \tau)$
- 13: $\tau \leftarrow \tau + \delta$; $\rho_r \leftarrow \rho_r - \delta$; $e_r \leftarrow e_r + \delta$
- 14: **if** $\rho_r = 0$ **then**
- 15: $C_r \leftarrow \tau$; $r \leftarrow \perp$
- 16: **end if**
- 17: **end while**

V. EXPERIMENTAL SETUP

A. Trace and Task Set

The workload is the `batch_task` table of the Alibaba cluster-trace-v2018, an eight-day trace of a production cluster of about 4000 machines running co-located online services and batch jobs [9], [50]. Each row is one task with the fields task name, instance count, job name, task type, status, start time, end time, planned CPU (100 per core) and normalised planned memory. The first 10,000 tasks in arrival order that satisfied the filters of Section IV were retained; they span approximately the first 24 hours of the trace.

B. Chronological Split

The task set was split at 80% in arrival order, giving 8,012 training and 1,988 test tasks. Because many tasks share an identical arrival timestamp, the boundary was advanced to the next distinct arrival time rather than cutting a tied cluster, and the run aborts unless the latest training arrival strictly precedes the earliest test arrival. No random or shuffled split was used anywhere in the pipeline.

C. Features

Table I lists every model input with the reason it is available when the task arrives. No other input was used.

D. Predictor Configuration

Gradient boosting used 300 trees of maximum depth 4, learning rate 0.05, row and column subsampling of 0.8, minimum child weight 5, the squared-error objective on the logarithmic target, and `random_state` equal to the seed below. This configuration was selected among four candidates on a chronological validation split formed from the last 20% of

TABLE I
MODEL INPUTS AND WHY EACH IS KNOWN AT TASK ARRIVAL.

Feature	Availability at arrival
requested_cpu	Planned CPU declared in the task's own submitted record (<code>plan_cpu</code>).
requested_mem	Planned memory declared in the same record (<code>plan_mem</code>).
n_instances	Instance count declared in the same record (<code>instance_num</code>).
hour_of_day	$\lfloor (a_i \bmod 86400) / 3600 \rfloor$, a function of a_i alone.
day_of_week	$\lfloor a_i / 86400 \rfloor \bmod 7$, a function of a_i alone.
previous_runtime	Runtime of the latest same-job task in $\mathcal{H}_i$ of (4); the global running mean if there is none.
rolling_mean_3	Mean of up to three latest same-job runtimes in $\mathcal{H}_i$; same fallback.
has_history	Whether $\mathcal{H}_i$ contains a same-job task; a property of past completions only.

the training set; the test set was not consulted. Linear regression used the scikit-learn ordinary-least-squares solver [52] behind a standardisation step, without regularisation. Exponential averaging used $\alpha = 0.5$ in (3); Last2 used a window of two; the median baseline used the median of the training runtimes. Both regression models and the median baseline were fitted on the training rows for which the historical features are defined; rows that arrived before any task had completed were dropped from fitting only, and the run aborts if any such row appears in the test split.

E. Schedulers and Sensitivity Sweeps

Seven schedulers were compared: first-come-first-served, round robin with a quantum of 4 simulated seconds, and SRTF driven by each of exponential averaging, Last2, linear regression, gradient boosting and the oracle. The sensitivity sweeps replace $\hat{t}_i$ by the true runtime perturbed by a noise model and floored at 1s: bounded uniform, $\hat{t}_i = t_i(1 + U(-\varepsilon, \varepsilon))$ with $\varepsilon \in \{0, 0.1, 0.2, 0.3, 0.5, 0.75, 1, 1.5, 2, 3, 5\}$, and log-normal, $\hat{t}_i = t_i \exp(\sigma Z)$ with $Z \sim \mathcal{N}(0, 1)$ and $\sigma \in \{0, 0.25, 0.5, 1, 1.5, 2, 3\}$. For each sweep the crossover is the smallest level at which SRTF's mean waiting time exceeds that of round robin, and the run aborts unless the zero-noise level reproduces oracle SRTF exactly.

F. Random Seeds, Software and Hardware

A single declared seed, 42, is used as the XGBoost `random_state` and as the base of the sweep generators, which use $42 + k$ (uniform) and $42 + 100 + k$ (log-normal) for the k -th level. Preprocessing, feature construction, ordinary least squares and the simulator are deterministic. The software stack was Python 3.13.14, NumPy 2.5.3, pandas 3.0.5, scikit-learn 1.9.1, XGBoost 3.4.1 and Matplotlib 3.11.2, run as a single process on an Intel Core Ultra 9 275HX (24 cores) with 15.5 GB of memory under Windows 11. Wall-clock cost of inference and simulation was not measured and is not charged against the simulated scheduler.

VI. RESULTS AND DISCUSSION

Table II reports the prediction error of every predictor on the test split, Table III the scheduling outcomes of the seven schedulers, and Table IV the two sensitivity sweeps. Fig. 1 plots average waiting time by scheduler against the oracle ceiling, Fig. 2 plots both sweeps against the round-robin reference, and Fig. 3 plots the gradient-boosted predictions against the actual runtimes. By (5), the gradient-boosted predictor recovers 58.5% of the waiting-time gap between exponential-averaging SRTF and oracle SRTF. The uniform sweep first exceeds the round-robin waiting time at $\varepsilon = 1.5$ and the log-normal sweep at $\sigma = 2$; the empirical standard deviation of $\log(\hat{t}_i/t_i)$ over the test split is 1.72 for gradient boosting and 2.21 for linear regression.

A. Prediction Error Does Not Predict Scheduling Quality

The three regression-style predictors, the constant median, linear regression and gradient boosting, are within 2.92% of one another in mean absolute error, and their root mean squared errors are nearly identical (Table II). Both metrics are dominated by a handful of very long tasks: the median test runtime is 6 s, but the longest test task runs for 104,558 s. Exponential averaging and Last2 are worse still. Almost every test task belongs to a job with no completed task, so both fall back to the running mean of completed runtimes, around 202 s, for nearly every task.

The scheduling outcomes separate these predictors sharply. Linear regression and gradient boosting differ by less than 2.92% in mean absolute error, yet SRTF driven by linear regression waits 1.33 times longer (Table III). The difference lies in ordering. SRTF uses only the order of the predictions, and the Spearman correlation with actual runtime is 0.66 for gradient boosting but only 0.09 for linear regression and 0.14 for exponential averaging. Mean absolute error rewards predicting the typical task well; a scheduler needs the order of the tasks right, especially at the long end. This is why the recovery percentage of (5), not mean absolute error, is the quantity reported as the headline.

B. Scheduling Outcomes

Oracle SRTF has the lowest mean waiting time of all seven schedulers, as it must, and first-come-first-served the highest. Replacing exponential averaging with gradient boosting reduces SRTF's mean waiting time from 63.2 h to 29.6 h, recovering 58.5% of the gap to the oracle. Round robin, however, waits only 11.5 h, and no prediction-driven SRTF comes close to it. Round robin pays for this with two orders of magnitude more context switches, and its response time is lower than that of every other policy because every task receives a quantum soon after it arrives.

Two properties of the test split shape these outcomes. First, every test task arrives within a window of 75 s, while the backlog takes far longer to drain, so the problem is close to ordering a single batch. Second, for linear regression, gradient boosting and the oracle, the number of context switches equals the number of tasks minus one, and waiting time equals

response time. No task was ever preempted once started, so SRTF reduced to non-preemptive shortest-predicted-job-first over that batch. Under these conditions the cost of a mistake is simple. A long task that is predicted to be short runs to completion ahead of every task ranked behind it.

Fig. 3 shows exactly this failure. Gradient-boosted predictions saturate: the largest prediction on the test split is 493 s. The 11 test tasks that run longer than 10,000 s receive predictions between 10.6 s and 264 s, indistinguishable from those of ordinary tasks. Their records at arrival do not tell them apart from typical tasks.

C. Sensitivity to Prediction Error

Table IV and Fig. 2 report SRTF under synthetic prediction error. Under bounded uniform noise SRTF is remarkably robust: its waiting time rises only slowly up to $\varepsilon = 1$, then exceeds round robin at $\varepsilon = 1.5$. The mechanism is the floor at 1 s. Once $\varepsilon > 1$ the factor $1 + U(-\varepsilon, \varepsilon)$ can be non-positive, some long tasks are predicted at 1 s, and each of them blocks the processor. The waiting time at $\varepsilon = 5$ is lower than at $\varepsilon = 3$ because each level is a single random draw; which long tasks happen to be floored varies between draws.

The log-normal model produces multiplicative errors of the kind a fitted model makes, and SRTF degrades smoothly under it, crossing round robin at $\sigma = 2$. The empirical log-error spread of gradient boosting, $\sigma = 1.72$, lies inside the region where SRTF still beats round robin, and random log-normal error of that spread yields about 10.2 h of waiting by linear interpolation of the sweep. The fitted model’s actual waiting time is 29.6 h, about 2.9 times higher. Linear regression’s spread, $\sigma = 2.21$, lies beyond the crossover.

The difference is structure. Random error of a given spread is equally likely to over- or under-predict any task, whereas the fitted model’s error is correlated with runtime: it systematically under-predicts the long tasks whose placement dominates mean waiting time. Structured error of a given spread therefore hurts SRTF far more than random error of the same spread. The practical consequence is that a deployment decision cannot be made from an error budget alone; it also depends on how the error correlates with the runtime itself.

D. Under-Prediction and the Fixed Estimate

Under (2) an under-predicted task’s predicted remaining service reaches zero while it still has work left, after which no arrival can preempt it. The under-prediction rate of Table II is 28.1% for gradient boosting, 48.3% for linear regression and 18.5% for exponential averaging. A low rate is not sufficient on its own: exponential averaging under-predicts least of the three, yet schedules worst, because its near-constant predictions carry almost no ordering information. What matters is whether the *long* tasks are under-predicted, and for gradient boosting they all are. The underlying design flaw is the rule of predicting once and never revising. A deployed predictor should increase its estimate once a task’s elapsed service exceeds it, as backfilling schedulers already do [23]. This is the concrete engineering recommendation of this study.

TABLE II
PREDICTION ERROR ON THE CHRONOLOGICAL TEST SET. THE UNDER-PREDICTION RATE IS THE FRACTION OF TEST TASKS WHOSE PREDICTED RUNTIME IS BELOW THE ACTUAL RUNTIME.

Predictor	MAE (s)	RMSE (s)	Under-prediction (%)
Median	415	4960	40.2
ExpAvg	523	4950	18.5
Last2	523	4950	18.5
Linear	414	4960	48.3
GBM	403	4950	28.1
Oracle	0	0	0

TABLE III
SCHEDULING OUTCOMES ON THE CHRONOLOGICAL TEST SET. TIMES ARE AVERAGES PER TASK.

Scheduler	Waiting (h)	Turnaround (h)	Response (h)	Context switches
FCFS	167	167	167	1,987
RR(q=4)	11.5	11.6	0.885	208,226
SRTF+ExpAvg	63.2	63.3	63.2	1,988
SRTF+Last2	63.2	63.3	63.2	1,987
SRTF+Linear	39.4	39.5	39.4	1,987
SRTF+GBM	29.6	29.7	29.6	1,987
SRTF+Oracle	5.78	5.89	5.78	1,987

VII. THREATS TO VALIDITY

Arrival-time proxy. The `batch_task` table records no submission time, so arrival is the recorded start time. Any queueing delay before a task started is invisible, and “arrival” here means “became runnable”. The causal ordering of historical features and the temporal features are all defined relative to this proxy.

Prediction target. The target is task-level runtime, the duration of one stage of a job executed by many instances. It is neither a CPU burst between I/O operations, which (3) was designed to estimate, nor strictly a whole job. The simulator treats it as a single CPU demand.

Short slice and batch-like test set. The 10,000 tasks span about 24 hours, and every test task arrives within 75 s. The `hour_of_day` and `day_of_week` features therefore act as indicators of position in the slice rather than of any daily or weekly cycle, although they are among the gradient-boosted model’s most important features. The near-simultaneous arrivals also mean that preemption plays almost no role in the test period, so the results characterise batch ordering more than online preemptive scheduling.

Single trace, simulation and uncharged inference. All results come from one trace and a simulator. Preemption cost is not modelled, the cost of running a predictor is not charged against scheduling latency, and no kernel implementation was attempted. The results may not transfer to other workloads.

Single noise draws. Each sensitivity level uses one random draw, so individual points carry sampling noise, as the non-monotone uniform sweep at its largest levels shows. The crossover levels should be read as approximate.

Weak learned models. On this slice the learned models barely improve on a constant median in mean absolute error.

TABLE IV
SENSITIVITY OF SRTF TO INJECTED PREDICTION ERROR. PREDICTIONS ARE THE TRUE RUNTIME PERTURBED BY THE STATED NOISE MODEL, FLOORED AT 1 S. AVERAGE WAITING TIME IS PER TASK.

Injected error level	Waiting (h)
<i>Bounded uniform:</i> $\hat{t} = t(1 + U(-\varepsilon, \varepsilon))$	
$\varepsilon = 0$	5.78
$\varepsilon = 0.1$	5.78
$\varepsilon = 0.2$	5.80
$\varepsilon = 0.3$	5.82
$\varepsilon = 0.5$	5.94
$\varepsilon = 0.75$	6.16
$\varepsilon = 1$	7.20
$\varepsilon = 1.5$	27.7
$\varepsilon = 2$	48.2
$\varepsilon = 3$	95.2
$\varepsilon = 5$	82.0
<i>Log-normal:</i> $\hat{t} = t \exp(\sigma Z)$, $Z \sim \mathcal{N}(0, 1)$	
$\sigma = 0$	5.78
$\sigma = 0.25$	5.88
$\sigma = 0.5$	6.15
$\sigma = 1$	7.25
$\sigma = 1.5$	8.93
$\sigma = 2$	11.9
$\sigma = 3$	25.1

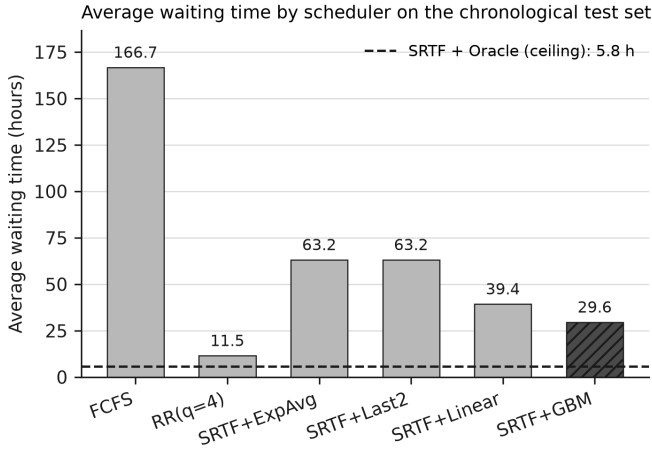


Fig. 1. Average waiting time per task by scheduler on the chronological test split. The dashed line is the oracle-driven SRTF ceiling.

The recovery percentage must therefore be read together with the round-robin comparison, not instead of it.

VIII. CONCLUSION AND FUTURE WORK

This paper measured how much of the waiting-time advantage of clairvoyant SRTF a learned, arrival-time runtime predictor can recover on a production cluster trace. A gradient-boosted model recovers 58.5% of the gap between exponential-averaging SRTF and oracle SRTF, but it does not make SRTF competitive with round robin, which uses no prediction at all. The reason is that the information SRTF needs most, which of the arriving tasks are the rare very long ones, is not present in the arrival-time features of this trace. Mean absolute error hides this failure, while rank correlation and the scheduling outcome expose it. The sensitivity analysis

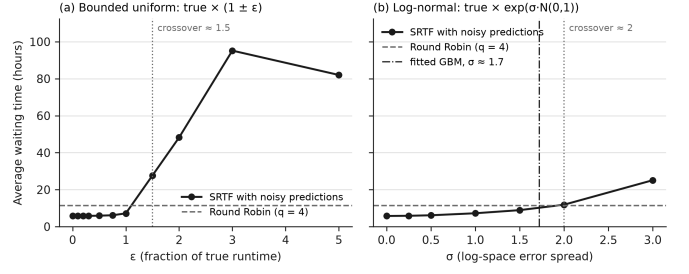


Fig. 2. Average waiting time of SRTF under injected prediction error: (a) bounded uniform, (b) log-normal. The dashed line is round robin; the dash-dotted line in (b) marks the empirical log-error spread of the fitted gradient-boosted model.

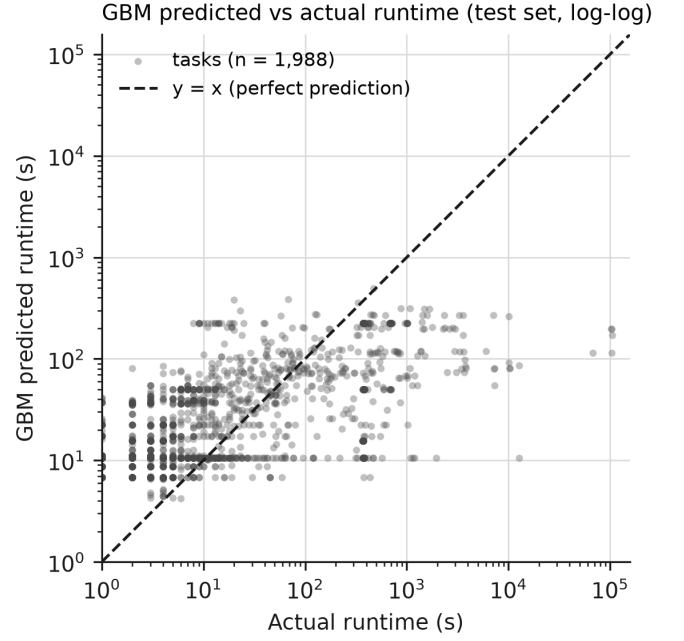


Fig. 3. Gradient-boosted predicted versus actual runtime on the test split (log-log). The dashed diagonal is perfect prediction.

shows that the fitted model’s error, being concentrated on long tasks, costs several times more waiting than random error of the same spread.

The specific percentages are properties of this trace slice and will not transfer. The methodology should: the recovery metric, the causality tests for features and online baselines, the oracle-equivalence test for the simulator, and the contrast between structured and random error apply to any trace.

Three directions follow directly from the results. First, predictions should be revised once a task’s elapsed service exceeds them, which closes the failure mode in which an under-predicted long task cannot be preempted. Second, the evaluation should be repeated on a slice spanning several days and on a second trace, such as the Google 2019 trace or GWA-T-4, so that temporal features can capture genuine periodicity and arrivals are spread out in time. Third, quantile or classification predictors that target the question “is this task

one of the very long ones?” address the decision the scheduler actually makes more directly than mean-error regression does.

REFERENCES

- [1] L. Schrage, “A proof of the optimality of the shortest remaining processing time discipline,” *Operations Research*, vol. 16, no. 3, pp. 687–690, 1968.
- [2] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [3] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, “The case for learned index structures,” in *SIGMOD 2018*, 2018, arXiv:1712.01208.
- [4] G. Vietri, L. V. Rodriguez, W. A. Martinez, S. Lyons, J. Liu, R. Rangaswami, M. Zhao, and G. Narasimhan, “Driving cache replacement with ML-based LeCaR,” in *HotStorage 2018*, 2018.
- [5] L. V. Rodriguez, F. Yusuf, S. Lyons, E. Paz, R. Rangaswami, J. Liu, M. Zhao, and G. Narasimhan, “Learning cache replacement with Cacheus,” in *USENIX FAST 2021*, 2021.
- [6] Z. Shi, X. Huang, A. Jain, and C. Lin, “Applying deep learning to the cache replacement problem,” in *MICRO 2019*, 2019.
- [7] Z. Song, D. S. Berger, K. Li *et al.*, “Learning relaxed Belady for content distribution network caching,” in *NSDI 2020*, 2020, pp. 529–544.
- [8] Z. Wang and M. O’Boyle, “Machine learning in compiler optimization,” *Proceedings of the IEEE*, vol. 106, no. 11, pp. 1879–1901, 2018.
- [9] Y. Cheng, Z. Chai, and A. Anwar, “Characterizing co-located datacenter workloads: An Alibaba case study,” in *APSys 2018*, 2018, arXiv:1808.02919.
- [10] P. A. Dinda, “A prediction-based real-time scheduling advisor,” in *IPDPS 2002*, 2002, pp. 88–95.
- [11] R. Wolski, “Dynamically forecasting network performance using the network weather service,” *Cluster Computing*, vol. 1, no. 1, pp. 119–132, 1998.
- [12] R. Wolski, N. Spring, and J. Hayes, “Predicting the CPU availability of timeshared Unix systems on the computational grid,” *Cluster Computing*, vol. 3, no. 4, pp. 293–301, 2000.
- [13] L. Yang, I. T. Foster, and J. M. Schopf, “Homeostatic and tendency-based CPU load predictions,” in *IPDPS 2003*, 2003.
- [14] Y. Zhang, W. Sun, and Y. Inoguchi, “Predict task running time in grid environments based on CPU load predictions,” *Future Generation Computer Systems*, vol. 24, no. 6, pp. 489–497, 2008.
- [15] T. Helmy, S. Al-Azani, and O. Bin-Obaidallah, “A machine learning-based approach to estimate the CPU-burst time for processes in the computational grids,” in *AIMS 2015*. IEEE, 2015.
- [16] S. Jha and R. K. Goyal, “CPU burst-time estimation using machine learning,” Feb. 2022.
- [17] A. Pourali *et al.*, “A fuzzy-based scheduling algorithm for prediction of next CPU-burst time to implement shortest process next.”
- [18] E. Effah, S. J. Atsu, Z. A. Brew *et al.*, “Predicting CPU burst times with ML to enhance shortest job first (SJF) and shortest remaining time first (SRTF) CPU scheduling,” 2025.
- [19] A. A. Shafin and K. M. Ahmed, “Leakage-safe and scheduler-aware machine learning for grid job runtime prediction,” arXiv preprint arXiv:2609.13701, 2026.
- [20] C. B. Lee, Y. Schwartzman, J. Hardy, and A. Snavey, “Are user runtime estimates inherently inaccurate?” in *JSSPP 2004*, 2004, pp. 253–263.
- [21] C. B. Lee and A. Snavey, “On the user-scheduler dialogue: studies of user-provided runtime estimates and utility functions,” *International Journal of High Performance Computing Applications*, vol. 20, no. 4, pp. 495–506, 2006.
- [22] S.-H. Chiang, A. Arpaci-Dusseau, and M. K. Vernon, “The impact of more accurate requested runtimes on production job scheduling performance,” in *JSSPP 2002*, 2002, pp. 103–127.
- [23] D. Tsafir, Y. Etsion, and D. G. Feitelson, “Backfilling using system-generated predictions rather than user runtime estimates,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 18, no. 6, pp. 789–803, 2007.
- [24] E. Gaussier *et al.*, “Improving backfilling by using machine learning to predict running times,” in *SC 2015*, 2015.
- [25] X. Chen, H. Zhang, H. Bai, C. Yang, X. Zhao, and B. Li, “Runtime prediction of high-performance computing jobs based on ensemble learning,” in *HP3C 2020*, 2020, pp. 56–62.
- [26] F. Chen, “Job runtime prediction of HPC cluster based on PC-transformer,” *The Journal of Supercomputing*, vol. 79, no. 17, pp. 20208–20234, 2023.
- [27] H. Cheon, J. Ryu, J. Ryou, C. Y. Park, and Y.-S. Han, “ARED: Automata-based runtime estimation for distributed systems using deep learning,” *Cluster Computing*, vol. 26, no. 5, pp. 2629–2641, 2023.
- [28] X. Chen, C.-D. Lu, and K. Pattabiraman, “Predicting job completion times using system logs in supercomputing clusters,” in *DSN-W 2013*, 2013, pp. 1–8.
- [29] M. Tanash, B. Dunn, D. Andresen, W. Hsu, H. Yang, and A. Okanlawon, “Improving HPC system performance by predicting job resources via supervised machine learning,” in *PEARC 2019*, 2019.
- [30] Y. Fan *et al.*, “Tobit-model-based runtime adjustment balancing accuracy against under-estimation rate.”
- [31] “Machine learning predictions for underestimation of job runtime on HPC system.” Springer, 2017.
- [32] V. Chlumský and D. Klusáček, “Improving accuracy of walltime estimates in PBS professional using soft walltimes,” in *JSSPP 2022*, 2022, pp. 192–210.
- [33] “UARP: Uncertainty-aware runtime prediction for preventing scheduler termination under wallclock constraints in HPC,” *The Journal of Supercomputing*, 2026.
- [34] H. Xiao *et al.*, “TARE: Tail-aware evaluation of HPC job runtime prediction,” arXiv preprint arXiv:2607.04935, 2026.
- [35] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *HotNets 2016*, 2016, pp. 50–56.
- [36] Y. Ye, X. Ren, J. Wang *et al.*, “A new approach for resource scheduling with deep reinforcement learning,” arXiv preprint arXiv:1806.08122.
- [37] “Data centers job scheduling with deep reinforcement learning,” in *PAKDD 2020*, 2020, arXiv:1909.07820.
- [38] G. Domeniconi *et al.*, “CuSH: Cognitive scheduler for heterogeneous high-performance computing system.”
- [39] R. de Freitas Cunha and L. Chaimowicz, “On the impact of MDP design for reinforcement learning agents in resource management,” arXiv preprint arXiv:2109.03202.
- [40] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *SIGCOMM 2019*, 2019.
- [41] D. Zhang *et al.*, “RLScheduler: An automated HPC batch job scheduler using reinforcement learning,” in *SC 2020*, 2020.
- [42] C. Reiss, J. Wilkes, and J. L. Hellerstein, “Google cluster-usage traces: Format + schema,” Google, Tech. Rep., 2011, revised 2014.
- [43] Y. Chen, A. Ganapathi, R. Griffith, and R. Katz, “Analysis and lessons from a publicly available Google cluster trace,” University of California, Berkeley, Tech. Rep., 2010.
- [44] C. Reiss, A. Tumanov, G. R. Ganger, R. H. Katz, and M. A. Kozuch, “Heterogeneity and dynamicity of clouds at scale: Google trace analysis,” in *SoCC 2012*, 2012.
- [45] Z. Liu and S. Cho, “Characterizing machines and workloads on a Google cluster,” in *ICPPW 2012*. IEEE, 2012.
- [46] A. Verma, L. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *EuroSys 2015*, 2015.
- [47] C. Lu *et al.*, “Imbalance in the cloud: An analysis on Alibaba cluster trace,” in *IEEE BigData 2017*, 2017.
- [48] Y. Cheng, A. Anwar, and X. Duan, “Analyzing Alibaba’s co-located datacenter workloads,” in *IEEE BigData 2018*, 2018, pp. 292–297.
- [49] Q. Liu and Z. Yu, “The elasticity and plasticity in semi-containerized co-locating cloud workload: A view from Alibaba trace,” in *SoCC 2018*, 2018.
- [50] “Characterizing co-located workloads in Alibaba cloud datacenters,” 2020.
- [51] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *KDD 2016*, 2016, pp. 785–794.
- [52] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.